NYU TANDON SCHOOL OF ENGINEERING

NYU TANDON CAREER HUB · FALL 2026

Data Science Bootcamp

Week 4 · SQL Fundamentals

Instructor: **Angelin Ann Jacob**

Section 1 · Tue Oct 20

Section 2 · Thu Oct 22

Useful Links

Zoom
All sessions are virtual. Use ONLY **your NYU SSO Login to be considered for attendance**

Brightspace
Assignments, submissions and the discussion board, monitored Monday to Thursday.

GitHub
Slides and Colab notebooks for every session.

Email
datasciencebootcamp@nyu.edu for questions, extensions and attendance approvals.

1:1 coaching
Optional 30-minute technical interview prep. Book it through Handshake.

NYU TANDON SCHOOL OF ENGINEERING

SQL Fundamentals

Ask a database a question, get a table back!

Data Science Bootcamp · Fall 2026



LEGO Database

github.com/shalakasaraogi/lego-database-inventory-management

Dataset name: LEGO Database, 8 connected tables (2017 snapshot compiled by Rachael Tatman)

Data Provided By:
Rebrickable, a community catalog of every official LEGO set

Size: 11,673 sets, 614 themes, 135 colors, 580,251 part rows, 1950 to 2017
Visit: rebrickable.com/downloads

HOW TO READ TODAY'S SESSION

Predict, code, share

PREDICT

Guess before you run

Post your guess in the Zoom chat, then run the cell and compare.

How many sets have more than 5,000 parts?

Which theme has the most sets?

YOUR TURN

Write the query, then share

Post your answer in the Zoom chat.

```
Hint    open it if you are stuck
Answer  open it only to check
```

EXAMPLE

Story: Which set has the most parts?

```
SELECT name FROM sets
ORDER BY num_parts DESC LIMIT 1
```

SQL: ask a question, get a table back

YOUR QUESTION

Which LEGO sets have more than 5,000 parts?

THE SQL QUERY

```
SELECT name, num_parts
FROM sets
WHERE num_parts > 5000
```

THE ANSWER: A TABLE

name	parts
Taj Mahal	5,922
Star Wars Mosaic	5,461
Window Bag	5,200

STRUCTURED QUERY LANGUAGE

SQL

Say "sequel" or "S-Q-L".

The standard language for data in databases.

YOU SAY WHAT, NOT HOW

pandas: a recipe of steps

filter → group → sort

SQL: a description of the result

"give me these columns, from this table, where..."

ONE LANGUAGE, MANY DATABASES

SQLite | PostgreSQL | MySQL | BigQuery | Snowflake | SQL Server

INTRODUCTION · WHY DATABASES?

One big sheet repeats itself; tables don’t

ONE BIG SHEET

name	year	theme
Statue of Liberty	2000	**Sculptures**
Eiffel Tower 1:300 Scale	2007	**Sculptures**
Taj Mahal	2008	**Sculptures**
Tower Bridge	2010	**Sculptures**
Sydney Opera House	2013	**Sculptures**

Sculptures is typed in every row: 23 times in the real data.

One typo, "Sculpture", and your counts are wrong.

TWO LINKED TABLES

sets

name	year	theme_id
Taj Mahal	2008	**276**
Tower Bridge	2010	**276**
Sydney Opera House	2013	**276**

themes

id	name
276	**Sculptures**

Stored once, linked by its id.

Fix a name in one place, every set sees it.

INTRODUCTION · WHERE SQL FITS

NumPy, pandas, SQL: one question, three tools

NUMPY · ARRAY

5922	4295	5200	4166	4023	4640
0	1	2	3	4	5

numbers by position
one data type, in memory

SETS WITH 5,000+ PARTS

```
parts[parts > 5000]
```

PANDAS · DATAFRAME

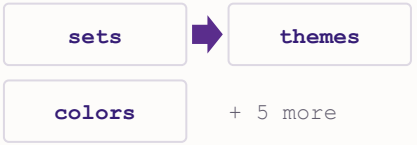
name	year	parts
Taj Mahal	2008	5922
Big Ben	2016	4166

one table, named columns
mixed types, in memory

SETS WITH 5,000+ PARTS

```
sets[sets["num_parts"] > 5000]
```

SQL · DATABASE



many tables, linked by keys
lives in a database, any size

SETS WITH 5,000+ PARTS

```
SELECT * FROM sets
WHERE num_parts > 5000
```

What is a relational database?

sets

set_num	theme_id	name	year
10189-1	276	Taj Mahal	2008
10214-1	276	Tower Bridge	2010
10179-1	174	Millennium Falcon - UCS	2007



themes

id	name
174	Star Wars Episode 4/5/6
276	Sculptures

Relational = tables related to each other through keys.

`abc` **TABLE**
one kind of thing: sets, themes

`abc` **ROW (record)**
one item: the Taj Mahal set

`abc` **COLUMN (field)**
one property of every row: year

`id` **PRIMARY KEY**
a unique id for each row: set_num

`fk` **FOREIGN KEY**
points to a key in another table

NYU TANDON SCHOOL OF ENGINEERING

Today's plan: from one table to many

1 · Databases

Tables, keys and the LEGO map. Loading the data.

2 · Filter

SELECT, WHERE, LIKE, NULL, ORDER BY. Data Story 1.

3 · Summarize

GROUP BY, HAVING, CASE WHEN, counting with conditions. Data Story 2.

4 · Join

INNER, LEFT and self joins, checking row counts. Data Stories 3 and 4.

5 · Windows

CTEs (WITH), ranking, top N per group, LAG, running totals.

6 · Together

A query built step by step, and the order SQL runs in.

SECTION 1 · RELATIONAL DATABASES

Eight tables, linked by keys



23 sets point to one theme row: **Sculptures**
Store once, link many times.

PRIMARY KEY

`sets.set_num`

one unique value per row

FOREIGN KEY

`sets.theme_id → themes.id`

points to a row in another table

THE TOOL

```
SELECT name, year
FROM sets
LIMIT 5   -- say what, not how
```

SECTION 2 · ANATOMY OF A QUERY

Read a query like a sentence

`SELECT name, year`	Show me the name and year columns	required
`FROM sets`	from the sets table	required
`WHERE year = 1950`	but only the rows from 1950,	optional
`ORDER BY name`	sorted A to Z by name,	optional
`LIMIT 5`	and just the first 5.	optional

Only **`SELECT`** and **`FROM`** are required. The others are optional, but always in this order.

SECTION 2 · HOW TO WRITE A QUERY

Six questions that write the query for you

1	What is one row of the answer?	PLAN FIRST
2	Which table has the data?	FROM / JOIN
3	Which rows do I need?	WHERE
4	Summarize per group?	GROUP BY
5	Keep only some groups?	HAVING
6	Which columns, order, how many?	SELECT, ORDER BY

QUESTION

What are the first six years after 2000, that had more than 500 sets, busiest first?

```
SELECT year, COUNT(*) AS n -- 6
FROM sets                  -- 2
WHERE year > 2000          -- 3
GROUP BY year              -- 4
HAVING COUNT(*) > 500      -- 5
ORDER BY n DESC            -- 6
```

ANSWER: ONE ROW PER YEAR

2014: 713	2015: 665	2012: 615	2016: 596

SECTION 2 · SELECTING AND FILTERING

Pick the columns, then keep the rows

SELECT keeps columns **WHERE** keeps rows

name	year	num_parts	theme_id
Taj Mahal	**2008**	**5,922**	276
Black Seas Barracuda	1989	915	148
Window Exploration Bag	**2010**	**5,200**	507
Train	1964	253	380
Millennium Falcon - UCS	**2007**	**5,195**	174
Big Ben	2016	4,166	43

Real rows from sets. Only 4 of 11,673 sets have more than 5,000 parts.

THE TOOL

```
SELECT name, year, num_parts
FROM sets
WHERE num_parts > 5000
-- 4 sets
```

WHERE TOOLKIT

`= <> >=`	`AND OR`	`BETWEEN`
`IN (...)`	`LIKE '%Train'`	`IS NULL`

Text goes in **'single quotes'**.
Missing values: **IS NULL**, never **= NULL**.

SECTION 2 · SQL, PANDAS AND NUMPY

WHERE is a boolean mask, written in words

QUERY	SQL	pandas	NumPy
Compare	`WHERE num_parts > 5000`	`sets[sets["num_parts"] > 5000]`	`parts[parts > 5000]`

Same idea as Week 2: only rows where the condition is True stay. SQL writes **AND**, pandas and NumPy write **&** with brackets.

SECTION 2 · SQL, PANDAS AND NUMPY

WHERE is a boolean mask, written in words

QUERY	SQL	pandas	NumPy
Compare	`WHERE num_parts > 5000`	`sets[sets["num_parts"] > 5000]`	`parts[parts > 5000]`
Both true	`WHERE year >= 2015 AND num_parts > 1000`	`sets[(sets["year"] >= 2015) & (sets["num_parts"] > 1000)]`	`parts[(years >= 2015) & (parts > 1000)]`

Same idea as Week 2: only rows where the condition is True stay. SQL writes **`AND`**, pandas and NumPy write **`&`** with brackets.

SECTION 2 · SQL, PANDAS AND NUMPY

WHERE is a boolean mask, written in words

QUERY	SQL	pandas	NumPy
Compare	`WHERE num_parts > 5000`	`sets[sets["num_parts"] > 5000]`	`parts[parts > 5000]`
Both true	`WHERE year >= 2015` `  AND num_parts > 1000`	`sets[(sets["year"] >= 2015) &` `  (sets["num_parts"] > 1000)]`	`parts[(years >= 2015) &` `  (parts > 1000)]`
In a range	`WHERE year` `  BETWEEN 1990 AND 1999`	`sets[sets["year"]` `  .between(1990, 1999)]`	`years[(years >= 1990) &` `  (years <= 1999)]`

Same idea as Week 2: only rows where the condition is True stay. SQL writes **`AND`**, pandas and NumPy write **`&`** with brackets.

SECTION 2 · SQL, PANDAS AND NUMPY

WHERE is a boolean mask, written in words

QUERY	SQL	pandas	NumPy
Compare	`WHERE num_parts > 5000`	`sets[sets["num_parts"] > 5000]`	`parts[parts > 5000]`
Both true	`WHERE year >= 2015 AND num_parts > 1000`	`sets[(sets["year"] >= 2015) & (sets["num_parts"] > 1000)]`	`parts[(years >= 2015) & (parts > 1000)]`
In a range	`WHERE year BETWEEN 1990 AND 1999`	`sets[sets["year"].between(1990, 1999)]`	`years[(years >= 1990) & (years <= 1999)]`
In a list	`WHERE year IN (1980, 1990)`	`sets[sets["year"].isin([1980, 1990])]`	`np.isin(years, [1980, 1990])`

Same idea as Week 2: only rows where the condition is True stay. SQL writes **AND**, pandas and NumPy write **&** with brackets.

SECTION 2 · SQL, PANDAS AND NUMPY

WHERE is a boolean mask, written in words

QUERY	SQL	pandas	NumPy
Compare	`WHERE num_parts > 5000`	`sets[sets["num_parts"] > 5000]`	`parts[parts > 5000]`
Both true	`WHERE year >= 2015 AND num_parts > 1000`	`sets[(sets["year"] >= 2015) & (sets["num_parts"] > 1000)]`	`parts[(years >= 2015) & (parts > 1000)]`
In a range	`WHERE year BETWEEN 1990 AND 1999`	`sets[sets["year"].between(1990, 1999)]`	`years[(years >= 1990) & (years <= 1999)]`
In a list	`WHERE year IN (1980, 1990)`	`sets[sets["year"].isin([1980, 1990])]`	`np.isin(years, [1980, 1990])`
Text pattern	`WHERE name LIKE '%Train'`	`sets[sets["name"].str.endswith("Train")]`	*no text tools*

Same idea as Week 2: only rows where the condition is True stay. SQL writes **AND**, pandas and NumPy write **&** with brackets.

SECTION 2 · SQL, PANDAS AND NUMPY

WHERE is a boolean mask, written in words

QUERY	SQL	pandas	NumPy
Compare	`WHERE num_parts > 5000`	`sets[sets["num_parts"] > 5000]`	`parts[parts > 5000]`
Both true	`WHERE year >= 2015 AND num_parts > 1000`	`sets[(sets["year"] >= 2015) & (sets["num_parts"] > 1000)]`	`parts[(years >= 2015) & (parts > 1000)]`
In a range	`WHERE year BETWEEN 1990 AND 1999`	`sets[sets["year"].between(1990, 1999)]`	`years[(years >= 1990) & (years <= 1999)]`
In a list	`WHERE year IN (1980, 1990)`	`sets[sets["year"].isin([1980, 1990])]`	`np.isin(years, [1980, 1990])`
Text pattern	`WHERE name LIKE '%Train'`	`sets[sets["name"].str.endswith("Train")]`	*no text tools*
Missing value	`WHERE parent_id IS NULL`	`themes[themes["parent_id"].isna()]`	`np.isnan(parent_ids)`

Same idea as Week 2: only rows where the condition is True stay. SQL writes **AND**, pandas and NumPy write **&** with brackets.

SECTION 2 · SORTING

ORDER BY sorts, LIMIT keeps the top

`FROM sets`

name	num_parts
Train	253
Taj Mahal	5,922
Big Ben	4,166
Black Seas Barracuda	915
Tower Bridge	4,295

`ORDER BY num_parts DESC`

name	num_parts
Taj Mahal	5,922
Tower Bridge	4,295
Big Ben	4,166
Black Seas Barracuda	915
Train	253

`LIMIT 3`

name	num_parts
Taj Mahal	**5,922**
Tower Bridge	**4,295**
Big Ben	**4,166**

ASC

smallest first (the default)

`1, 2, 3 · A to Z · old to new`

DESC

largest first

`3, 2, 1 · Z to A · new to old`

TWO COLUMNS

```
ORDER BY year DESC,
         num_parts DESC
-- sort by two columns
```

SECTION 2 · SQL, PANDAS AND NUMPY

Choose, create, sort and cut columns

TASK	SQL	pandas	NumPy
Pick columns	`SELECT name, year FROM sets`	`sets[["name", "year"]]`	`data[:, [1, 2]] # by position`
New column	`SELECT 2026 - year AS age`	`sets["age"] = 2026 - sets["year"]`	`age = 2026 - years`
Unique values	`SELECT DISTINCT year`	`sets["year"].unique()`	`np.unique(years)`
Sort, largest first	`ORDER BY num_parts DESC`	`sets.sort_values("num_parts", ascending=False)`	`np.sort(parts)[::-1]`
First 10 rows	`LIMIT 10`	`.head(10)`	`[:10]`

pandas and NumPy chain the steps you choose, in order. **SQL describes the finished table** and the database picks the steps.

SECTION 2 · SORTING · DATA STORY 1

The largest LEGO sets ever made



THE STORY

Which set has the most parts?

Taj Mahal: **5,922** parts, one of only 4 sets above 5,000.

THE TOOL

```
SELECT name, year, num_parts
FROM sets
ORDER BY num_parts DESC
LIMIT 10
```

READ THE ROWS

2 of the top 3 are bags of bricks.

SECTION 3 · SUMMARIZING

Many rows in, one number out

AGGREGATE FUNCTIONS

num_parts
5,922
4,295
5,200
4,166
4,023
4,640

→

```
COUNT(*)  6
SUM       28,246
AVG       4,707.7
MIN       4,023
MAX       5,922
```

GROUP BY year: ONE ROW PER GROUP

year	num_parts
2008	5,922
2010	4,295
2010	5,200
2016	4,166
2016	4,023
2016	4,640

→

year	sets	avg
2008	1	5,922
2010	2	4,748
2016	3	4,276

`GROUP BY` = split into piles, summarize each pile. Same idea as **`.groupby()`** in pandas.

SECTION 3 · SQL, PANDAS AND NUMPY

GROUP BY is groupby()

TASK	SQL	pandas	NumPy
Count rows	`SELECT COUNT(*) FROM sets`	`len(sets)`	`parts.size`
Average	`SELECT AVG(num_parts)` `FROM sets`	`sets["num_parts"].mean()`	`parts.mean()`
Smallest, largest	`SELECT MIN(year), MAX(year)`	`sets["year"].min()` `sets["year"].max()`	`years.min()` `years.max()`

Count per group	`SELECT year, COUNT(*)`
	`FROM sets`
	`GROUP BY year`
		`sets.groupby("year").size()`	`np.unique(years,`
			`  return_counts=True)`

Average per group	`SELECT year,`
	`  AVG(num_parts)`
	`FROM sets GROUP BY year`
		`sets.groupby("year")`
		`  ["num_parts"].mean()`
			no groupby: use pandas

Split into groups, summarize each, combine. **Every SELECT column is grouped or aggregated**, like the columns you pass to groupby().

SECTION 3 · DATA STORY 2

Are LEGO sets getting bigger?

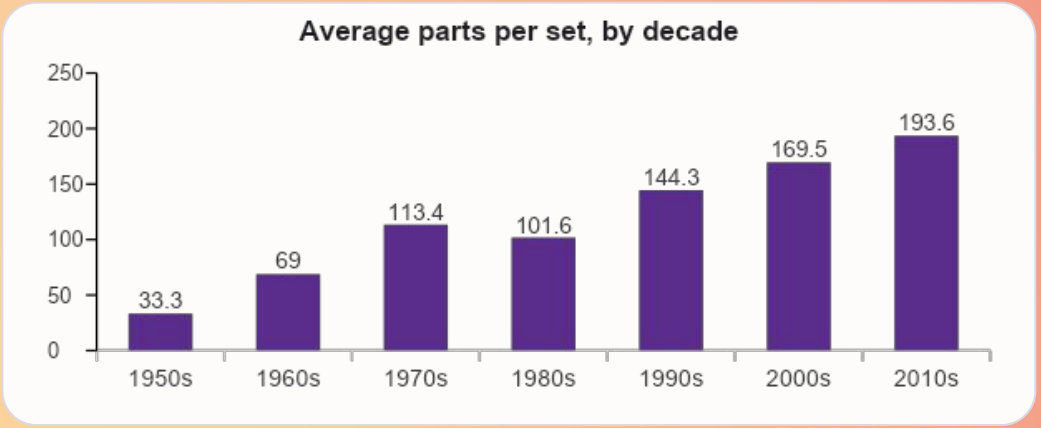
THE STORY

Were sets smaller when we were kids?

Adult fans say modern sets are much bigger.

THE TOOL

```
SELECT year,
  COUNT(*) AS sets,
  AVG(num_parts)
FROM sets
GROUP BY year
```



4,425

sets in the 2010s, up from 132 in the 1950s

SECTION 3 · SUMMARIZING

Filter groups, build categories

WHERE filters rows, before grouping

```
WHERE num_parts > 0
```

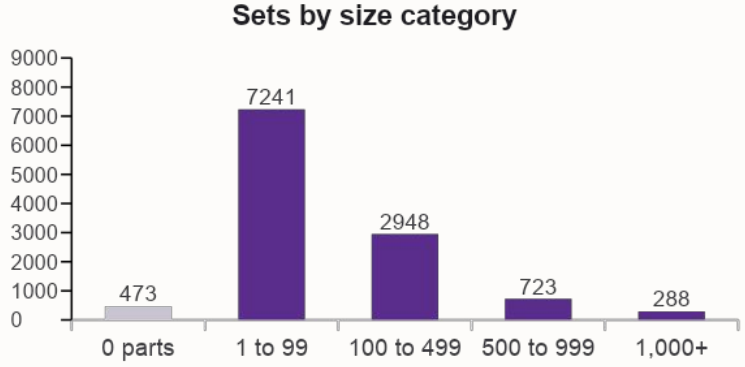
HAVING filters groups, after grouping

```
HAVING COUNT(*) > 500
```

YEARS WITH MORE THAN 500 SETS

2011 2012 2013 2014 2015 2016

Peak: 713 sets in 2014



THE TOOL

```
CASE WHEN num_parts < 100 THEN 'Small'
     WHEN num_parts < 500 THEN 'Medium'
     ELSE 'Large' END AS size
```

SECTION 3 · SQL, PANDAS AND NUMPY

HAVING and CASE WHEN, translated

TASK	SQL	pandas	NumPy

Keep big groups (HAVING)	`SELECT year, COUNT(*)`
	`FROM sets GROUP BY year`
	`HAVING COUNT(*) > 500`
		`n = sets.groupby("year").size()`
		`n[n > 500]`
			`yrs, n = np.unique(years,`
			`  return_counts=True)`
			`yrs[n > 500]`

Two labels	`CASE WHEN num_parts < 100`
	`  THEN 'Small'`
	`  ELSE 'Big' END`
		`np.where(sets["num_parts"] < 100,`
		`  "Small", "Big")`
			`np.where(parts < 100,`
			`  "Small", "Big")`

Many labels	`CASE WHEN num_parts < 100`
	`  THEN 'S'`
	`  WHEN num_parts < 500`
	`  THEN 'M' ELSE 'L' END`
		`pd.cut(sets["num_parts"],`
		`  [-1, 99, 499, np.inf],`
		`  labels=["S", "M", "L"])`
			`np.select(`
			`  [parts<100, parts<500],`
			`  ["S", "M"], "L")`

HAVING = filter the result of a groupby. **CASE WHEN** = np.where for two labels, np.select or pd.cut for more.

SECTION 3 · SUMMARIZING

Count only the rows you want

STEP 1 · LABEL EACH ROW 1 OR 0

name	num_parts	big
Taj Mahal	5,922	1
Train	253	0
Big Ben	4,166	1
Black Seas Barracuda	915	0
Fire Station	104	0
Tower Bridge	4,295	1

```
big = CASE WHEN num_parts >= 1000 THEN 1 ELSE 0 END
```

STEP 2
`SUM(big) = 3`

STEP 3
`COUNT(*) = 6`

STEP 4
`100.0 * 3 / 6 = 50.0%`

THE PATTERN

```
SELECT COUNT(*) AS all_colors,
       SUM(CASE WHEN is_trans = 't'
           THEN 1 ELSE 0 END) AS clear
FROM colors
-- 135 colors, 28 transparent
```

THE TRAP: WHOLE-NUMBER DIVISION

```
100 * 28 / 135      → 20
100.0 * 28 / 135    → 20.7
```

SQLite, PostgreSQL and SQL Server drop the decimals. MySQL keeps them.

SECTION 4 · COMBINING TABLES

JOIN: follow the key to the other table

sets AS s

name	theme_id
Taj Mahal	276
Tower Bridge	276
Millennium Falcon - UCS	174
Window Exploration Bag	507

themes AS t

id	name
174	Star Wars Episode 4/5/6
276	Sculptures
507	Educational and Dacta
36	Fire

Fire: no sets attached

result

set	theme
Taj Mahal	Sculptures
Tower Bridge	Sculptures
Millennium Falcon - UCS	Star Wars Ep. 4/5/6
Window Exploration Bag	Educational and Dacta

JOIN keeps rows that match in both

```
FROM sets AS s
JOIN themes AS t
  ON s.theme_id = t.id
```

LEFT JOIN keeps every left row

```
FROM themes AS t
LEFT JOIN sets AS s ON ...
WHERE s.set_num IS NULL   -- 39
```

SECTION 4 · SQL, PANDAS AND NUMPY

JOIN is merge() in pandas

TASK	SQL	pandas
Inner join	`SELECT * FROM sets AS s JOIN themes AS t ON s.theme_id = t.id`	`sets.merge(themes, left_on="theme_id", right_on="id")`
Left join	`SELECT * FROM themes AS t LEFT JOIN sets AS s ON t.id = s.theme_id`	`themes.merge(sets, how="left", left_on="id", right_on="theme_id")`
Find rows with no match	`... LEFT JOIN ... WHERE s.set_num IS NULL -- 39 themes`	`m = themes.merge(sets, ..., how="left", indicator=True) m[m["_merge"] == "left_only"]`

NumPy

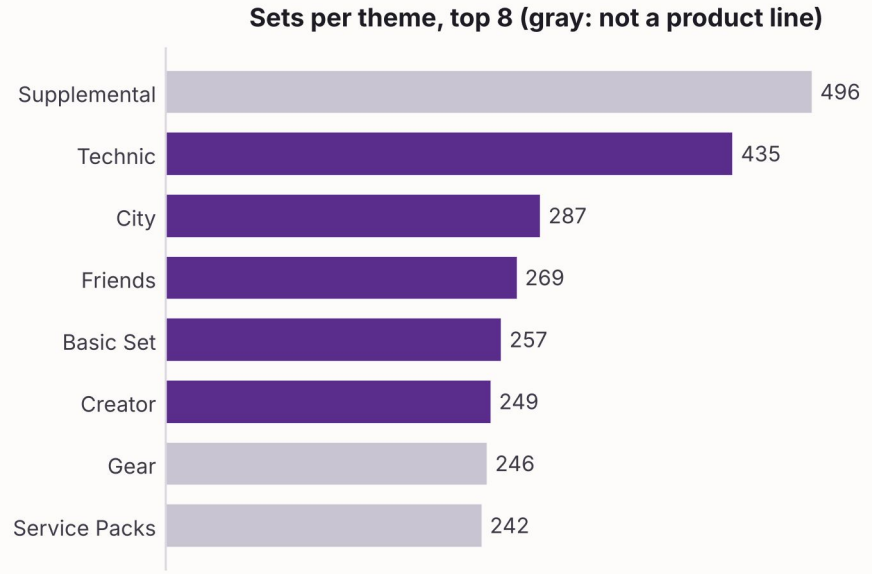
No tables, no keys, no joins.

NumPy works on one array at a time. Combining tables is where pandas take over.

`ON` = left_on / right_on. `LEFT JOIN` = how="left". Forget `ON` and every row matches every other row.

SECTION 4 · DATA STORY 3

Which theme has the most LEGO sets?



THE STORY

Star Wars, City or Technic?

Names live in themes, so we need a JOIN.

THE TOOL

```
FROM sets AS s
JOIN themes AS t
  ON s.theme_id = t.id
GROUP BY t.name
```

THE TWIST

No. 1 is spare bricks, not a product line.

SECTION 4 · SELF JOINS

A table joined to itself

themes AS child

id	name	parent_id
1	Technic	NULL
158	Star Wars	NULL
171	Ultimate Collector Series	158
174	Star Wars Episode 4/5/6	171
494	Friends	NULL

result

theme	parent_theme
Technic	**(top level)**
Star Wars	**(top level)**
Ultimate Collector Series	Star Wars
Star Wars Episode 4/5/6	Ultimate Collector Series
Friends	**(top level)**

```sql
SELECT child.name,
       COALESCE(parent.name, '(top level)')
FROM themes AS child
LEFT JOIN themes AS parent
  ON child.parent_id = parent.id
```

WHY EACH PIECE

Two aliases: one table, two roles.

`LEFT JOIN` keeps themes with no parent.

`COALESCE` swaps NULL for a label.

Interview classic: employees and managers.

SECTION 4 · CHECK YOUR JOINS

Watch the row count after a join

11,673

rows in sets, one per set

→

JOIN

```
JOIN inventories AS i
  ON s.set_num = i.set_num
```

→

11,681

rows after the join

WHERE THE EXTRA ROWS COME FROM

11,673	sets
−3	sets with no inventory drop out
+11	extra copies of 8 sets with 2 or more versions
11,681	rows: no new sets, just repeats

FIND THE DUPLICATES

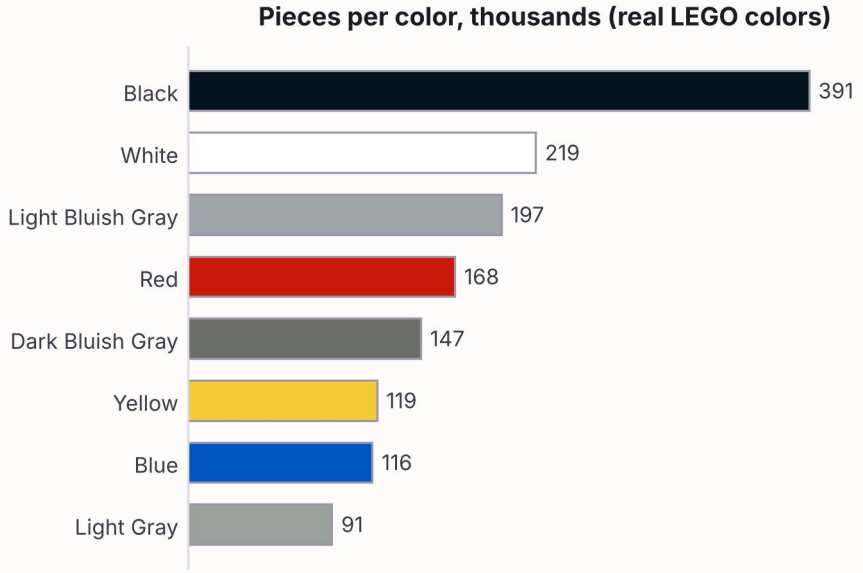
```
SELECT set_num, COUNT(*)
FROM inventories
GROUP BY set_num
HAVING COUNT(*) > 1   -- 8 sets
```

INTERVIEW HABIT

Compare row counts before and after every join.

SECTION 4 · DATA STORY 4

The colors of LEGO



THE STORY

Which color is used most?

Black: **391,036** pieces.

THE TOOL

```
FROM inventory_parts AS ip
JOIN colors AS c
  ON ip.color_id = c.id
SUM(ip.quantity)
```

THE TWIST

Palette grew from **15** to **117** colors.

SECTION 5 · CTES AND WINDOW FUNCTIONS

WITH: name a step, then use it

STEP 1 · NAME IT

```
WITH theme_summary AS (
  SELECT t.name    AS theme_name,
         COUNT(*) AS number_of_sets,
         AVG(s.num_parts)  AS average_parts
  FROM sets AS s JOIN themes AS t
    ON s.theme_id = t.id
  GROUP BY t.name
)
```

STEP 2 · USE IT

```
SELECT *
FROM theme_summary
WHERE number_of_sets >= 20
ORDER BY average_parts DESC
```

RESULT

Sculptures 1,717

Model 629

Star Wars Ep. 4/5/6 593

READS TOP TO BOTTOM

Each step has a name, like a variable in Python.

FILTER WITH PLAIN WHERE

number_of_sets is now a column, so WHERE works and no HAVING is needed.

NEEDED FOR WINDOWS

Window functions cannot go in WHERE, so we calculate them in a CTE and filter after.

SECTION 5 · CTES AND WINDOW FUNCTIONS

Window functions keep every row

GROUP BY year: ROWS COLLAPSE

year	set
2008	Taj Mahal
2010	Tower Bridge
2010	Window Exploration Bag
2016	Big Ben
2016	Death Star
2016	Firehouse Headquarters

→

year	sets
2008	1
2010	2
2016	3

6 rows in, 3 rows out

COUNT(*) OVER (PARTITION BY year): ROWS STAY

year	set	sets_that_year
2008	Taj Mahal	1
2010	Tower Bridge	2
2010	Window Exploration Bag	2
2016	Big Ben	3
2016	Death Star	3
2016	Firehouse Headquarters	3

6 rows in, 6 rows out, plus a new column

```
COUNT(*) OVER (  PARTITION BY year  ORDER BY num_parts DESC )
```

what to calculate — *the groups: nothing collapses* — *the order inside each group*

SECTION 5 · CTES AND WINDOW FUNCTIONS

Three ways to number a tie

THE SIX QUIETEST YEARS

year	sets	row_number	rank	dense_rank
1960	3	1	1	1
1953	**4**	**2**	**2**	**2**
1959	**4**	**3**	**2**	**2**
1950	7	4	4	3
1965	10	5	5	4
1964	11	6	6	5

1953 and 1959 both had 4 sets. The next row gets 4 with RANK but 3 with DENSE_RANK.

ROW_NUMBER()

1, 2, 3, 4: never a tie.

Who gets 2 among ties is arbitrary. Use: top N per group.

RANK()

1, 2, 2, 4: ties share, then a gap.

Use: rankings where ties matter.

DENSE_RANK()

1, 2, 2, 3: ties share, no gap.

Use: the second-highest value.

SECTION 5 · CTES AND WINDOW FUNCTIONS

Top N per group in three steps

STEP 1 · NUMBER EACH GROUP

```
ROW_NUMBER() OVER (
  PARTITION BY (year / 10) * 10
  ORDER BY num_parts DESC
) AS position
```

1 = the largest set in its decade

STEP 2 · WRAP IT IN A CTE

```
WITH ranked AS (
  SELECT name, year, ...,
         position
  FROM sets
)
```

WHERE cannot see a window result

STEP 3 · KEEP THE TOP

```
SELECT *
FROM ranked
WHERE position = 1
```

position <= 3 gives a top 3

RESULT: THE LARGEST SET OF EVERY DECADE

1950s	1960s	1970s	1980s	1990s	2000s	2010s
Gift Package	Town Plan	U.S.S. Constellation	Teachers Resource Set	Bonus Lego Basic Town	Taj Mahal	Window Exploration Bag
244	760	973	1,534	2,472	5,922	5,200

SECTION 5 · CTES AND WINDOW FUNCTIONS

Look back with LAG, add up with SUM OVER

SETS PER YEAR, WITH THREE WINDOW COLUMNS

year	sets	previous	change	running	3-yr avg
2012	615	503	+112	8,810	520.7
2013	593	615	−22	9,403	570.3
2014	713	593	+120	10,116	640.3
2015	665	713	−48	10,781	657.0
2016	596	665	−69	11,377	658.0
2017	296	596	−300	11,673	519.0

`previous` copies the row above. `running` adds every row so far: 11,673 by 2017. 2017's −300 is the incomplete year.

LOOK BACK

```
LAG(number_of_sets)
  OVER (ORDER BY year)
```

RUNNING TOTAL

```
SUM(number_of_sets)
  OVER (ORDER BY year)
```

MOVING AVERAGE

```
AVG(number_of_sets) OVER (
  ORDER BY year ROWS BETWEEN
  2 PRECEDING AND CURRENT ROW)
```

CAREFUL

LAG uses the previous row, not year. 1950 has no row above: NULL.

SECTION 6 · PUTTING IT TOGETHER

Build a query one line at a time

QUESTION Which themes got the most sets from 2010 on?

STEP 1 · TABLES

```
FROM sets AS s
JOIN themes AS t
 ON s.theme_id = t.id
```

→ 11,673 rows

STEP 2 · ROWS

```
+ WHERE s.year >= 2010
```

→ 4,425 rows

STEP 3 · GROUPS

```
+ GROUP BY t.name
SELECT t.name,
  COUNT(*) AS n
```

→ 204 themes

STEP 4 · TOP 5

```
+ ORDER BY n DESC
+ LIMIT 5
```

→ 5 rows

RESULT

Friends	Gear	Star Wars	Ninjago	Duplo
269	211	194	191	169

THE SAME QUESTION IN PANDAS

```
m = sets.merge(themes, left_on="theme_id",
               right_on="id", suffixes=("", "_t"))
(m[m["year"] >= 2010]
 .groupby("name_t").size().nlargest(5))
```

SECTION 6 · PUTTING IT TOGETHER

SQL runs in a different order than you write it

YOU WRITE	SELECT	FROM	WHERE	GROUP BY	HAVING	ORDER BY	LIMIT
SQL RUNS	1 FROM	2 WHERE	3 GROUP BY	4 HAVING	5 SELECT	6 ORDER BY	7 LIMIT

SELECT runs fifth, so **`WHERE`** cannot use **`COUNT(*)`** or a window function: they do not exist yet.

WHY WINDOWS NEED A CTE

```
-- fails: the window is not ready yet
... WHERE ROW_NUMBER() OVER (...) = 1
-- works: number first, filter after
WITH ranked AS (SELECT ..., ROW_NUMBER()
  OVER (...) AS position FROM sets)
SELECT * FROM ranked WHERE position = 1
```

SPOT THE BUG

```
SELECT year, COUNT(*)
FROM sets
WHERE COUNT(*) > 500
GROUP BY year
```

Error: misuse of aggregate. Fix: move it to **`HAVING`**.

SECTION 6 · DRAWING INSIGHTS

What we learned today

THE INSIGHT	THE EVIDENCE
The biggest sets are not always models	Taj Mahal: **5,922** parts; 2 of the top 3 are bags of bricks
LEGO makes more sets, and bigger ones	**132** sets in the 1950s, **4,425** in the 2010s; average size **33** to **194** parts
Always check where the data ends	**2017** has only **296** sets: the snapshot stops mid-year
Black is LEGO's most used color	**391,036** pieces; the palette grew from **15** to **117** colors

NYU TANDON SCHOOL OF ENGINEERING

Before next week

Take-home

Exercises 1 to 5 in today's notebook, including interview practice. Submit on Brightspace within one week, by 11:59 PM.

Feedback form

Fill it out after today's session. It counts toward your credential.

Week 5: Presentations

Mid-program project presentations and feedback. Tue Oct 27 / Thu Oct 29.